

Financial Agent Harness

Sistem multi-agent untuk riset dan evaluasi saham berbasis evidence terlacak dan debat argumentatif (Bull vs Bear), dengan Judge Agent sebagai pengambil keputusan akhir.

/judge BBKA

Daftar Isi

01	Ringkasan Eksekutif	3
02	Latar Belakang & Masalah yang Diselesaikan	4
03	Prinsip Desain Utama	5
04	Arsitektur Sistem	6
05	Evidence Store & Manajemen Context	8
06	Bull, Bear & Debate Agent	9
07	Judge Agent & Metodologi Skor	10
08	Command System (Harness)	11
09	Agent Ekstensi (Optional Agent)	12
10	Strategi Model Routing	13
11	Desain UI/UX	14
12	Manajemen Risiko & Mitigasi	15
13	Roadmap Implementasi (4 Minggu)	17
14	Disclaimer & Positioning	18
15	Kesimpulan	19

BAGIAN 01

Ringkasan Eksekutif

Financial Agent Harness adalah kerangka kerja multi-agent untuk riset dan evaluasi saham, dengan **/judge** sebagai command andalan: sebuah alur riset otomatis yang berpuncak pada argumentasi Bull vs Bear yang diadili oleh Judge Agent — bukan sekadar model tunggal yang melempar output "BUY" atau "SELL".

Perbedaan mendasar dibanding pendekatan "LLM + API data" yang umum: setiap klaim yang dihasilkan sistem ini tertaut ke evidence ID yang bisa ditelusuri balik ke data mentahnya. Pengguna tidak hanya melihat skor akhir, tapi juga bagaimana skor itu terbentuk — argumen mana yang kuat, argumen mana yang lemah, dan data apa yang mendasarinya.

Dokumen ini menjabarkan arsitektur sistem, desain tiap agent, metodologi skor, strategi manajemen context dan biaya, daftar risiko teknis beserta mitigasinya, serta roadmap implementasi 4 minggu menuju hackathon.

Empat pilar desain

Evidence-first (setiap klaim punya sumber) · Structured intermediate output (bukan teks bebas antar-agent) · Debate terbatas (argumentasi terkontrol, bukan obrolan tanpa akhir) · Auditability (proses bisa ditelusuri, bukan black box).

BAGIAN 02

Latar Belakang & Masalah yang Diselesaikan

Kebanyakan alat analisis saham berbasis LLM bekerja dengan pola sederhana: ambil data, masukkan ke satu prompt besar, minta model menyimpulkan. Pola ini punya tiga kelemahan yang ingin dihindari oleh Financial Agent Harness:

1. Output tidak bisa diaudit

Kesimpulan seperti "BBCA layak dibeli" tidak menjelaskan argumen mana yang mendasarinya, apalagi data mana yang mendukung argumen tersebut. Pengguna harus percaya begitu saja pada model.

2. Context cepat penuh

Mengirim data mentah (ratusan baris finansial, transaksi, berita) langsung ke satu prompt membuat context window cepat habis, dan justru membuat model kesulitan fokus pada sinyal yang relevan.

3. Kesimpulan tidak teruji dari sudut berlawanan

Model tunggal yang diminta "beri opini" cenderung menghasilkan analisis satu arah tanpa benar-benar mempertimbangkan argumen tandingan yang kuat.

Financial Agent Harness menjawab ketiganya lewat: Evidence Store sebagai lapisan data terpisah dari context LLM, output terstruktur berbasis claim-evidence, dan mekanisme Bull vs Bear yang dipaksa saling menyerang argumen sebelum Judge mengambil keputusan.

BAGIAN 03

Prinsip Desain Utama

Evidence-first

Setiap angka atau pernyataan yang dipakai sebuah agent harus punya evidence ID yang bisa ditelusuri ke Evidence Store. Tidak ada klaim "mengambang" tanpa sumber.

Jangan kirim raw data antar-agent

Data mentah (100+ baris data finansial, puluhan sinyal pasar) tidak pernah dikirim langsung ke agent lain. Data mentah disimpan di Evidence Store; agent hanya menerima Research Brief — ringkasan terstruktur dengan referensi sumber per angka.

Structured intermediate output

Antar-agent berkomunikasi lewat objek terstruktur (claim, evidence_ids, confidence) — bukan paragraf naratif bebas. Ini membuat context lebih kecil dan lebih mudah divalidasi secara otomatis.

Debate terbatas

Bull dan Bear tidak berdebat tanpa batas. Maksimum dua ronde (Bull → Bear → Bull Rebuttal → Bear Final), dan ronde kedua hanya berjalan bila ada sinyal konflik yang cukup kuat untuk dilanjutkan — bukan default yang selalu dijalankan.

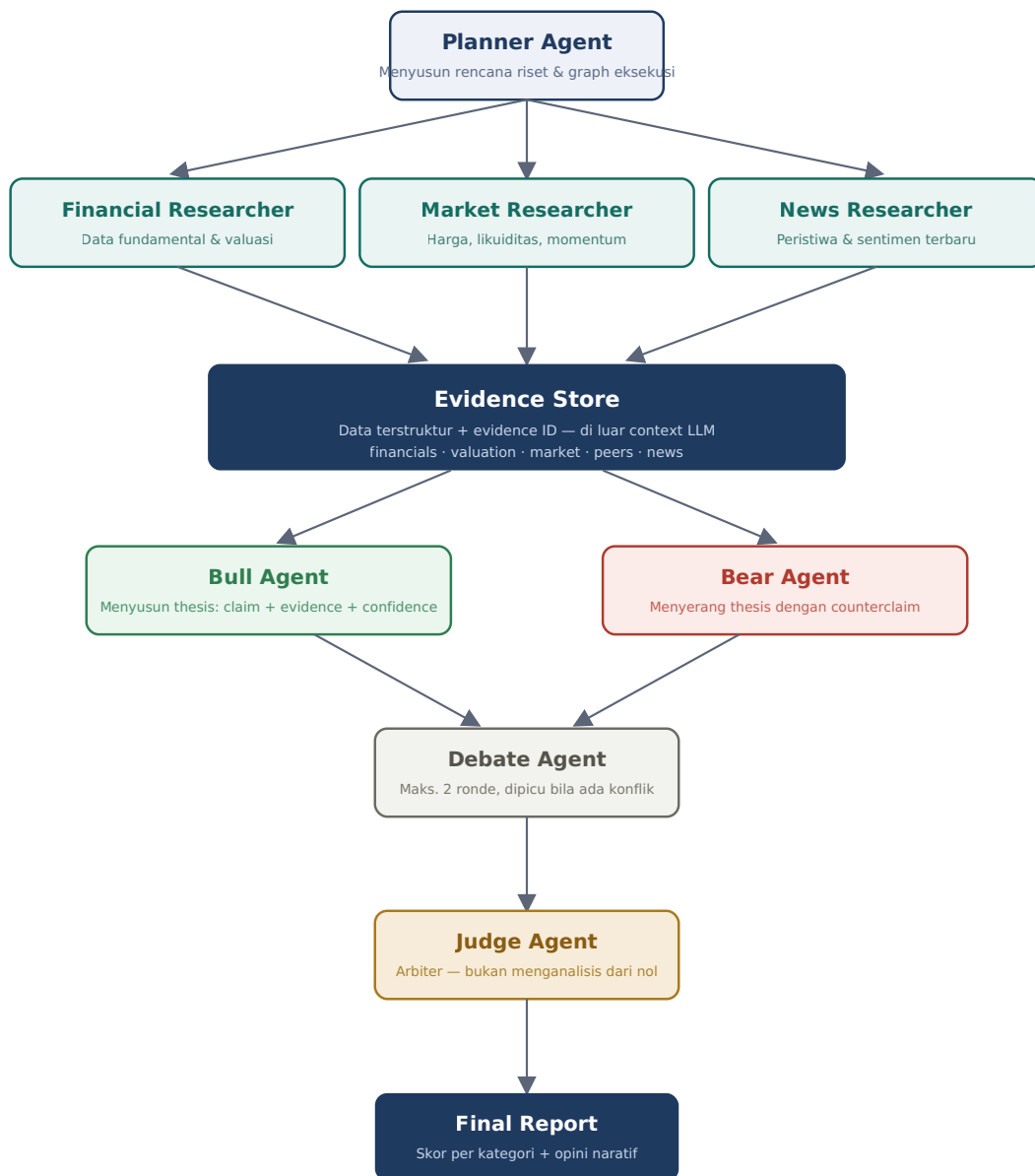
Retrieval, bukan broadcast

Judge Agent tidak menerima seluruh riwayat percakapan. Ia meminta (retrieve) evidence spesifik yang relevan dengan klaim yang sedang dinilai.

BAGIAN 04

Arsitektur Sistem

Diagram berikut menggambarkan alur eksekusi default untuk command **/judge**. Tiga agent riset berjalan paralel, hasilnya dikonsolidasikan ke Evidence Store, lalu diteruskan ke lapisan argumentasi (Bull/Bear/Debate) sebelum diadili oleh Judge.



Gambar 1 — Execution graph default untuk command **/judge**

Peran tiap komponen

Komponen	Peran	Input	Output
Planner	Memecah tujuan riset jadi task & menyusun execution graph	Ticker, command yang dipanggil	Rencana task + graph eksekusi
Researcher (×3)	Mengambil & merangkum data per domain	Task dari Planner, akses data/API	Data terstruktur ke Evidence Store
Evidence Store	Menyimpan data mentah & ringkasan, memberi evidence ID	Output ketiga researcher	Research Brief per permintaan agent
Bull / Bear	Menyusun thesis & counterthesis berbasis evidence	Research Brief	Claim + evidence_ids + confidence
Debate	Mengatur pertukaran argumen terbatas	Thesis Bull & Bear	Rebuttal & respons final
Judge	Mengadili argumen & menghitung skor akhir	Evidence, thesis, hasil debat	Skor per kategori + opini naratif

BAGIAN 05

Evidence Store & Manajemen Context

Evidence Store adalah lapisan state yang hidup di luar context LLM — bisa berupa Postgres/ Supabase — yang menyimpan seluruh data riset per ticker beserta metadata sumber dan periode. Agent tidak pernah menerima isi Evidence Store secara penuh; mereka menerima Research Brief yang sudah disaring.



Gambar 2 — Raw data tidak pernah langsung menyentuh context Bull/Bear

Contoh Research Brief

Setiap angka dalam brief membawa referensi sumber dan periode, sehingga bisa divalidasi otomatis maupun ditelusuri manual oleh pengguna lewat UI.

Metrik	Nilai	Sumber (evidence_id)	Periode
ROE	23.1%	sectors.financial_snapshot	FY2026
Revenue growth	12.4%	sectors.financial_snapshot	FY2026
PE Ratio	18.2	sectors.valuation	FY2026
PE peer median	14.7	sectors.peer_comparison	FY2026

Catatan penting: beda model tidak otomatis menghemat context

Mengganti model per agent (mis. Bull pakai model A, Bear pakai model B) tidak mengurangi total token — tiap agent tetap menerima input penuh sesuai isi promptnya. Penghematan nyata datang dari empat hal: Evidence Store eksternal, output terstruktur, context compression (ringkasan bertingkat), dan retrieval selektif (agent meminta hanya evidence yang relevan).

BAGIAN 06

Bull, Bear & Debate Agent

Bull Agent

Bull tidak mengeluarkan opini bebas ("BBCA bagus"), melainkan menyusun thesis dalam struktur claim → evidence → confidence:

Claim	Evidence	Confidence
Profitabilitas kuat	ROE 23.1%, ROA 3.4%	Strong
Trajektori laba tetap positif	Growth laba bersih 8.7% YoY	Moderate

Bear Agent

Bear menerima Research Brief *plus* thesis Bull, lalu menyerang claim tertentu dengan counterclaim berbasis evidence miliknya sendiri — bukan sekadar menyangkal.

Claim yang diserang	Counterargument	Kekuatan
"BBCA valuasinya menarik"	PE 18.2 berada di atas median peer 14.7	High

Debate Agent

Bull dan Bear tidak dibiarkan berdebat tanpa batas — itu boros token dan sulit dikendalikan. Alurnya dibuat linear dan berbatas:

Bull Thesis → Bear Response → Bull Rebuttal → Bear Final → STOP (maks. 2 ronde)

Ronde kedua tidak berjalan otomatis. Judge/Planner mengevaluasi apakah ronde pertama sudah menghasilkan cukup sinyal konflik untuk diadili, atau perlu satu ronde lagi.

BAGIAN 07

Judge Agent & Metodologi Skor

Judge tidak membuat analisis dari nol. Ia bertindak sebagai arbiter atas evidence, thesis Bull, thesis Bear, dan hasil debat — lalu merangkum argumen mana yang kuat dan mana yang lemah, sebelum menghitung skor.

Kenapa skor tidak boleh jadi black box

Angka akhir seperti "82/100" tanpa penjelasan bobot akan melemahkan seluruh prinsip auditability yang dibangun di lapisan evidence. Karena itu bobot kategori ditetapkan secara eksplisit di kode (deterministik), sementara Judge hanya mengisi skor per kategori berikut justifikasinya. Agregasi akhir dihitung oleh sistem, bukan ditebak bebas oleh LLM.

Rubrik bobot kategori

Kategori	Bobot	Dasar Penilaian
Financial Health	25%	ROE, ROA, margin, kualitas neraca
Growth	20%	Pertumbuhan revenue & laba, konsistensi
Valuation	20%	PE/PBV relatif terhadap peer & historis
Market Momentum	20%	Kinerja harga, likuiditas, tren
Risk	15%	Volatilitas, konsentrasi risiko, sentimen negatif
Overall	100%	Rata-rata tertimbang, dihitung deterministik

Struktur output Judge

- **Ringkasan argumen** — jumlah claim Bull/Bear, berapa yang kuat vs lemah
- **Argumen paling meyakinkan dan counterargument terkuat**
- **Skor per kategori** + skor keseluruhan (dihitung deterministik dari bobot di atas)
- **Opini naratif** — misalnya *"Bullish, namun valuasi membatasi potensi kenaikan"* — bukan sekadar label BUY/SELL

BAGIAN 08

Command System (Harness)

Financial Agent Harness dirancang sebagai platform command, bukan fitur tunggal. Setiap command memakai execution graph yang berbeda namun berbagi Evidence Store yang sama.

Command	Deskripsi	Agent Graph	Prioritas Build
/judge	Evaluasi investasi lengkap satu ticker	Planner → Riset → Bull/Bear → Debate → Judge	MVP — Minggu 1-2
/compare	Bandingkan beberapa ticker sejenis	Planner → Peer Research → Valuasi → Judge	MVP — Minggu 4
/research	Riset mentah tanpa opini/skor	Planner → Riset → Ringkasan	Roadmap
/investigate	Menyelidiki pergerakan harga tak biasa	Planner → Market → News → Anomaly	Roadmap
/challenge	Menguji satu klaim spesifik dari pengguna	Planner → Riset terarah → Debate	Didesain, belum diimplementasi
/screen	Menyaring saham berdasar kriteria bebas	Planner → Screening → Ranking	Didesain, belum diimplementasi

Command bertanda "Didesain, belum diimplementasi" tetap ditunjukkan dalam pitch sebagai bukti arsitektur yang ekstensibel, tanpa perlu berfungsi penuh saat demo.

BAGIAN 09

Agent Ekstensi (Optional Agent)

Agent inti selalu aktif; agent tambahan bersifat opt-in lewat flag, misalnya `/judge BBCA --with dividend risk`. Ini menjaga default run tetap cepat dan murah, sekaligus menunjukkan ekstensibilitas arsitektur.

Agent Inti (selalu aktif)	Agent Opsional (toggle)
Financial Researcher	Dividend Analyst
Market Researcher	Insider Activity Analyst
News Researcher	Sector Analyst
Bull / Bear Agent	Macro Analyst
Judge Agent	Technical Analyst
	Risk Analyst
	ESG Analyst
	Valuation Specialist

Untuk hackathon: implementasikan interface plug-in generik + 1–2 contoh agent opsional (mis. Dividend Analyst) sebagai bukti konsep. Sisanya cukup terdaftar di roadmap.

BAGIAN 10

Strategi Model Routing

Tidak semua agent membutuhkan model paling mahal. Tugas ekstraksi data terstruktur cocok untuk model cepat/murah; tugas argumentasi dan penilaian membutuhkan model dengan reasoning lebih kuat.

Agent / Tugas	Kebutuhan	Tier Model	Alasan
Planner	Dekomposisi task sederhana	Kecil / cepat	Tugas terstruktur, tidak butuh reasoning dalam
Researcher (×3)	Ekstraksi & normalisasi data	Cepat	Volume tinggi, tugas repetitif
Bull / Bear	Argumentasi berbasis evidence	Reasoning	Butuh kualitas argumen yang tajam
Judge	Sintesis & keputusan akhir	Reasoning terkuat tersedia	Titik keputusan paling kritis di seluruh pipeline

Untuk hackathon

Mulai dengan dua tier saja (satu model murah, satu model reasoning) agar tidak menghabiskan waktu men-debug perbedaan format tool-calling/JSON mode antar-provider. Tambah provider ketiga hanya bila waktu tersisa di Minggu 4.

BAGIAN 11

Desain UI/UX

UI diposisikan sebagai "AI research IDE", bukan dashboard finance biasa — nilai jualnya justru terletak pada transparansi proses, bukan hanya hasil akhir.

Panel	Isi
Kiri — Run	Daftar agent dalam graph eksekusi beserta status (selesai / berjalan / menunggu)
Tengah — Live Activity	Narasi langkah agent yang sedang berjalan (mis. argumen Bull secara real-time) dengan tautan "Lihat evidence"
Kanan — Research Data	Tabel data mentah per domain, terhubung ke evidence ID yang direferensikan di tengah

Tab audit

Overview · Financials · Market · News · Peers · Evidence · Debate · Final Verdict — memungkinkan pengguna menelusuri sendiri bagaimana sebuah keputusan terbentuk, dari data mentah hingga skor akhir.

BAGIAN 12

Manajemen Risiko & Mitigasi

Risiko	Dampak Bila Diabaikan	Mitigasi
Evidence hallucination	Claim mengacu evidence_id yang tidak ada / tidak relevan — merusak kredibilitas audit trail	Validator deterministik (bukan LLM) yang mengecek evidence_id benar-benar ada & nilainya cocok, sebelum claim diteruskan ke tahap berikutnya
Confidence score tidak terkalibrasi	Angka desimal (0.91) memberi ilusi presisi statistik yang sebenarnya tidak ada	Ganti ke kategori (Strong/Moderate/Weak) berbasis aturan objektif, mis. jumlah metrik independen yang mendukung claim
Biaya per run tinggi	10+ pemanggilan LLM per / judge; mahal bila dijalankan berulang	Model murah untuk Planner/Researcher; cache Evidence Store per ticker/hari; ronde debat kedua bersifat kondisional
Latensi pipeline	Rantai sekuensial (riset → debat → judge) terasa lambat	Paralelkan tiga researcher; streaming status ke UI; pre-fetch ticker populer sebelum demo
Scope creep	Mengejar seluruh roadmap sekaligus berisiko tidak ada yang benar-benar solid saat deadline	Satu pipeline inti (/ judge) solid dulu; command & agent lain cukup didesain sebagai roadmap
Skor jadi black box	Angka akhir tanpa penjelasan bobot melemahkan prinsip auditability	Bobot kategori eksplisit & deterministik (Bagian 07); Judge hanya mengisi skor per kategori, bukan angka akhir bebas
Non-determinism	Ticker sama dijalankan dua kali bisa menghasilkan verdict berbeda	Temperature rendah pada Judge; cache hasil per snapshot data, bukan generate ulang tiap request
Bias urutan pada Judge	Kecenderungan condong ke argumen yang disebut terakhir dalam debat	Beri Judge rubrik/checklist eksplisit, bukan ruang reasoning bebas tanpa struktur
Ketergantungan data eksternal	Rate limit / downtime API data pasar merusak demo live	Caching agresif + data fallback offline khusus untuk sesi presentasi
Risiko regulasi	Skor + opini untuk saham publik berbatasan dengan rekomendasi investasi (ranah OJK di Indonesia)	Disclaimer eksplisit di UI & laporan: "untuk riset & edukasi, bukan nasihat keuangan"

Risiko	Dampak Bila Diabaikan	Mitigasi
Prompt injection dari data eksternal	Konten berita hasil scraping bisa membawa instruksi tersembunyi	Perlakukan seluruh konten eksternal sebagai data, bukan instruksi, sebelum masuk context agent lain

BAGIAN 13

Roadmap Implementasi (4 Minggu)

Minggu	Fokus	Deliverable
1	Spine	Planner + orchestration graph (DAG engine); skema Evidence Store; satu sumber data terintegrasi (mis. Sectors API) ke Financial Researcher; alur kasar Planner → Financial Researcher → Bull tunggal, tersimpan di Evidence Store (UI boleh masih JSON/CLI)
2	Debate + Judge + validasi	Market & News Researcher paralel; Bull + Bear + Debate 1 ronde; Judge dengan rubrik skor eksplisit (Bagian 07); evidence validator deterministik (wajib, bukan opsional); confidence jadi kategorikal
3	UI + robustness	Live UI (execution graph + tab audit), memakai library graph siap pakai; caching Evidence Store per ticker/hari; ronde debat kedua dibuat dinamis (bukan selalu fix); fallback data offline untukantisipasi demo live
4	Perluasan roadmap + polish	Implementasi penuh / compare (reuse Evidence Store); command lain didesain di arsitektur & didemokan sebagai "next"; 1–2 contoh agent opsional sebagai bukti konsep; model routing 2 tier; dokumentasi & skrip demo cadangan

BAGIAN 14

Disclaimer & Positioning

Bukan nasihat keuangan

Financial Agent Harness dirancang sebagai alat riset dan edukasi yang menunjukkan bagaimana sebuah kesimpulan terbentuk dari data dan argumen — bukan sebagai rekomendasi investasi berizin. Skor dan opini dihasilkan oleh AI dan dapat keliru. Disclaimer ini perlu ditampilkan secara eksplisit di UI dan setiap laporan yang dihasilkan sistem, sekaligus menjadi bagian dari nilai jual: pengguna diajak memeriksa sendiri evidence di balik setiap klaim, bukan sekadar menerima skor akhir.

BAGIAN 15

Kesimpulan

Financial Agent Harness dibangun di atas prinsip yang jarang ditemukan bersamaan dalam proyek hackathon sejenis: evidence yang bisa ditelusuri, argumentasi yang diuji dari dua sisi berlawanan, skor yang dapat dijelaskan bobotnya, dan arsitektur command yang bisa diperluas tanpa mengubah fondasi. Ini secara eksplisit memenuhi kriteria yang biasa dicari pada penilaian custom agent orchestration: multi-step reasoning, routing, state management, memory, eksekusi otonom, hingga antarmuka yang dirancang khusus untuk proses ini — bukan sekadar model bahasa yang dibungkus API data.

Dengan roadmap 4 minggu di atas, prioritas utamanya jelas: satu pipeline / judge yang benar-benar solid dan bisa diaudit, dengan sisa command dan agent opsional sebagai bukti bahwa arsitekturnya memang dirancang untuk tumbuh — bukan janji kosong di slide terakhir.

Dokumen ini adalah draft desain v1.0 dan disusun untuk mendukung pengembangan & submission hackathon. Detail implementasi (skema database, kontrak API antar-agent, format prompt) dapat dijabarkan lebih lanjut sebagai dokumen teknis terpisah.